/init

> "Analyse the project and write a summary, or create a default config: /init [config|help]" (SlashCommandDef, crates/ragent-tui/src/app/state.rs).

Overview

`/init` has two independent jobs. Bare `/init` (or any unknown argument) kicks off a one-shot project analysis: it posts an "Analysing project…" notice, injects an analysis prompt as a user message, and runs it on the general agent. The resulting analysis - languages, frameworks, architecture, entry points, build/test system, conventions, and key files - is saved into structured memory via `memory_store` so later sessions recall it. `/init config` instead writes a fresh default `ragent.json` to disk when you want a starting point to edit.

The handler is the `init` arm in `crates/ragent-tui/src/app/slash.rs` (approximately lines 2655-2827).

Syntax

```
/init
/init config
/init help
```

Options / Subcommands

Form	Description
`/init`	Run the one-shot project analysis on the general agent and store the result as a project-analysis memory
`/init config`	Write `Config::default()` as pretty JSON to `<config_dir>/ragent/ragent.json`
`/init help`	Show the subcommand summary

Examples

Analyse the current project

```
/init
```

Behaviour: a `Analysing project...` notice is posted, the analysis prompt is injected as a user message, and the run is dispatched to the general agent (one-shot, no agent-stack push, `is_processing = true`). The analysis prompt asks the model to inspect:

- languages and frameworks in use,
- overall architecture and entry points,
- the build and test system,
- project conventions,
- the key files a contributor should read first,

and then to persist the result with:

```
memory_store  category: "fact"  confidence: 0.8  tags: ["project-analysis"]
```

Write a default config file

```
/init config
```

Output (file did not exist):

```
From: /init config
[ok] Wrote default configuration to /home/user/.config/ragent/ragent.json
```

Output (file already exists):

```
From: /init config
[warn] Config file already exists, skipping: /home/user/.config/ragent/ragent.json
```

Distinct `[err]` messages are emitted if the config directory is missing, or if serialisation, directory creation, or the final write fails; the offending path is included in the message.

Show help

```
/init help
```

Output (subcommand summary).

Output

- Bare `/init` runs one analysis turn; its output is the agent's own response in the message window, plus the `Analysing project...` notice beforehand. The structured result lands in the memory store as a `fact` with confidence 0.8 and tag `project-analysis`.
- `/init config` writes pretty-printed JSON of `Config::default()` to `<config_dir>/ragent/ragent.json` (on Linux, `~/.config/ragent/ragent.json`). It never overwrites an existing file - it skips with `[warn]`.
- `/init help` prints the subcommand summary.

Related

- `/config show` - inspect the resolved configuration after editing
- `/config save` - back up the global config before manual edits
- `/reload` - pick up config changes in the running session
- `memory_recall` tool - query the stored project analysis